

```

1  /
    *****
    *****
2  MPLAB Harmony Application Source File
3
4  Company:
5      Microchip Technology Inc.
6
7  File Name:
8      app.c
9
10 Summary:
11     This file contains the source code for the MPLAB Harmony
    application.
12
13 Description:
14     This file contains the source code for the MPLAB Harmony
    application. It
15     implements the logic of the application's state machine and it may
    call
16     API routines of other MPLAB Harmony modules in the system, such as
    drivers,
17     system services, and middleware. However, it does not call any of
    the
18     system interfaces (such as the "Initialize" and "Tasks" functions)
    of any of
19     the modules in the system or make any assumptions about when those
    functions
20     are called. That is the responsibility of the configuration-
    specific system
21     files.
22
    *****
    *****/
23
24 // DOM-IGNORE-BEGIN
25 /
    *****
    *****
26 Copyright (c) 2013-2014 released Microchip Technology Inc. All rights
    reserved.
27
28 Microchip licenses to you the right to use, modify, copy and distribute
29 Software only when embedded on a Microchip microcontroller or digital
    signal
30 controller that is integrated into your product or third party product
31 (pursuant to the sublicense terms in the accompanying license
    agreement).
32
33 You should refer to the license agreement accompanying this Software
    for
34 additional information regarding your rights and obligations.
35

```

```
36 SOFTWARE AND DOCUMENTATION ARE PROVIDED "AS IS" WITHOUT WARRANTY OF ANY  ↗
    KIND,
37 EITHER EXPRESS OR IMPLIED, INCLUDING WITHOUT LIMITATION, ANY WARRANTY  ↗
    OF
38 MERCHANTABILITY, TITLE, NON-INFRINGEMENT AND FITNESS FOR A PARTICULAR  ↗
    PURPOSE.
39 IN NO EVENT SHALL MICROCHIP OR ITS LICENSORS BE LIABLE OR OBLIGATED  ↗
    UNDER
40 CONTRACT, NEGLIGENCE, STRICT LIABILITY, CONTRIBUTION, BREACH OF  ↗
    WARRANTY, OR
41 OTHER LEGAL EQUITABLE THEORY ANY DIRECT OR INDIRECT DAMAGES OR EXPENSES
42 INCLUDING BUT NOT LIMITED TO ANY INCIDENTAL, SPECIAL, INDIRECT,  ↗
    PUNITIVE OR
43 CONSEQUENTIAL DAMAGES, LOST PROFITS OR LOST DATA, COST OF PROCUREMENT  ↗
    OF
44 SUBSTITUTE GOODS, TECHNOLOGY, SERVICES, OR ANY CLAIMS BY THIRD PARTIES
45 (INCLUDING BUT NOT LIMITED TO ANY DEFENSE THEREOF), OR OTHER SIMILAR  ↗
    COSTS.
46
    *****  ↗
    *****/
47 // DOM-IGNORE-END
48
49
50 //  ↗
    *****  ↗
    *****
51 //  ↗
    *****  ↗
    *****
52 // Section: Included Files
53 //  ↗
    *****  ↗
    *****
54 //  ↗
    *****  ↗
    *****
55
56 #include "app.h"
57 #include "Mc32gestSpiFT800.h"
58 #include "Mc32Delays.h"
59
60
61
62 // === Initialisation DMA Channel 0 vers OC2RS ===
63
64
65 //  ↗
    *****  ↗
    *****
66 //  ↗
    *****  ↗
    *****
```

```
67 // Section: Global Data Definitions
68 //
        *****
        *****
69
70 //
        *****
        *****
71
72 //
        *****
        *****
73 /* Application Data
74
75     Summary:
76     Holds application data
77
78     Description:
79     This structure holds the application's data.
80
81     Remarks:
82     This structure should be initialized by the APP_Initialize
        function.
83
84     Application strings and buffers are defined outside this
        structure.
85 */
86
87 APP_DATA appData;
88
89 //
        *****
        *****
90 //
        *****
        *****
91 // Section: Application Callback Functions
92 //
        *****
        *****
93 //
        *****
        *****
94
95 /* TODO: Add any necessary callback functions.
96 */
97
98 //
        *****
        *****
99 //
        *****
        *****
```

```
100 // Section: Application Local Functions
101 //
102 //
103
104
105 /* TODO: Add any necessary local functions.
106 */
107
108
109 //
110 //
111 // Section: Application Initialization and State Machine Functions
112 //
113 //
114
115 /
116 Function:
117     void APP_Initialize ( void )
118
119 Remarks:
120     See prototype in app.h.
121 */
122
123 void APP_Initialize(void) {
124     /* Place the App state machine in its initial state. */
125     appData.state = APP_STATE_INIT;
126
127
128     /* TODO: Initialize your application's state machine and other
129      * parameters.
130      */
131 }
132
133 /
134 Function:
135     void APP_Tasks ( void )
136
```



```

184          //          wr32(RAM_DL + 12, END());
185          //          wr32(RAM_DL + 16, COLOR_RGB(160, 22, 22)); // change colour to red
186          //          wr32(RAM_DL + 20, POINT_SIZE (160)); // set point size to 20 pixels in radius
187          //          wr32(RAM_DL + 24, BEGIN(POINTS)); // start drawing points
188          //          wr32(RAM_DL + 28, VERTEX2II(192, 133, 0, 0)); // red point
189          //          wr32(RAM_DL + 32, END());
190          wr32(Addr: RAM_DL + 8, Value: DISPLAY()); // Marque fin de la display list
191          //          wr8(REG_DLSWAP, DLSWAP_FRAME);
192          //          while (rd8(REG_DLSWAP) != DLSWAP_DONE);
193
194          wr8(Addr: REG_DLSWAP, Value: DLSWAP_FRAME);
195          affich = 0;
196      }
197      wr8(Addr: REG_GPIO_DIR, Value: 0x80); // Met GPIO7 en sortie
198      wr8(Addr: REG_GPIO, Value: 0x80); // allume le backlight (met GPIO7 à 1)
199
200      //          APP_UpdateState(APP_STATE_WAIT);
201
202      break;
203  }
204  case APP_STATE_WAIT:
205
206      /* TODO: implement your application state machine.*/
207
208
209      /* The default state should never be executed. */
210  default:
211      {
212          /* TODO: Handle error in application's state machine. */
213          break;
214      }
215  }
216  }
217
218  void APP_UpdateState(APP_STATES NewState) {
219      appData.state = NewState;
220  }
221  /
222  End of File
223  */

```